

Documento 06 – Arquitetura Backend

Versão: 1.0

Status: Em elaboração

Capítulo 1 – Introdução

1.1 Objetivo

Este documento define a arquitetura técnica do backend do Gestor de Territórios e Publicações (GTP).

Seu propósito é detalhar a organização interna da aplicação, estabelecendo padrões de desenvolvimento, estrutura de pacotes, responsabilidades das camadas e diretrizes para implementação.

Todas as definições aqui apresentadas derivam do Documento 05 – Arquitetura do Sistema, aprofundando especificamente a solução baseada em Java 21 e Spring Boot 3.

1.2 Escopo

Este documento contempla:

organização do projeto backend;

arquitetura em camadas;

estrutura de pacotes;

organização por módulos;

componentes técnicos;

estratégias de persistência;

tratamento de exceções;

validações;

auditoria;

testes;

integração entre módulos.

Aspectos específicos de segurança, Docker e API REST serão detalhados nos Documentos 08, 09 e 10.

1.3 Objetivos Arquiteturais

O backend do GTP deverá atender aos seguintes objetivos:

Modularidade

Organizar o código por módulos de negócio, reduzindo acoplamento e facilitando manutenção.

Escalabilidade

Permitir a evolução do sistema com inclusão de novos módulos e funcionalidades sem necessidade de reestruturações significativas.

Manutenibilidade

Adotar convenções que tornem o código claro, consistente e de fácil compreensão.

Testabilidade

Facilitar a implementação de testes unitários, de integração e funcionais.

Segurança

Aplicar autenticação, autorização, auditoria e proteção de dados de forma integrada à arquitetura.

1.4 Tecnologias Adotadas

CategoriaTecnologiaLinguagemJava 21 (LTS)FrameworkSpring Boot
3.xBuildMavenPersistênciaSpring Data JPAORMHibernateBanco de DadosPostgreSQL
17MigraçõesFlywaySegurançaSpring SecurityAutenticaçãoJWTDocumentaçãoOpenAPI /
SwaggerMapeamentoMapStructRedução de CódigoLombokValidaçãoJakarta
ValidationTestesJUnit 5, Mockito, Testcontainers1.5 Princípios Arquiteturais

O backend seguirá os princípios definidos no Documento 05, aplicados especificamente à implementação.

Separação de responsabilidades: cada classe terá uma única responsabilidade.

Baixo acoplamento: dependências entre módulos serão minimizadas.

Alta coesão: classes relacionadas permanecerão agrupadas.

Inversão de dependência: interfaces serão priorizadas em relação a implementações.

Código limpo: nomenclatura clara, métodos pequenos e ausência de lógica duplicada.

Domínio independente: as regras de negócio não dependerão de frameworks.

1.6 Arquitetura Geral do Backend

A aplicação será organizada em camadas, seguindo os princípios de DDD e Clean Architecture.

API REST



Controllers



Use Cases



Domain Services



Repositories



PostgreSQL 17

Cada camada possuirá responsabilidades bem definidas, reduzindo dependências cruzadas.

1.7 Organização por Módulos

O backend será dividido em módulos correspondentes ao domínio do negócio.

gtp-backend



■■■ auth

■■■ congregation

■■■ people

- users
- territories
- publications
- orders
- campaigns
- notifications
- reports
- settings
- audit
- shared

Cada módulo será autônomo, contendo suas próprias entidades, casos de uso, controladores e repositórios.

1.8 Organização Interna dos Módulos

Cada módulo seguirá uma estrutura padronizada.

orders

-

- application

- ■■■ dto

- ■■■ mapper

- ■■■ service

- ■■■ usecase

-

- domain

- ■■■ entity

- ■■■ repository

- ■■■ service

- ■■■ event

-

- infrastructure

- ■■■ persistence

- ■■■ integration

-

- interfaces

- controller

- exception

Essa organização facilita a manutenção e a localização de responsabilidades.

1.9 Componentes Compartilhados

O módulo shared reunirá funcionalidades utilizadas por toda a aplicação.

Exemplos:

tratamento global de exceções;

utilitários;

configurações comuns;

classes base;

respostas padronizadas;

paginação;

filtros;

constantes.

1.10 Convenções Gerais

Para garantir consistência, serão adotadas convenções de nomenclatura e organização.

Exemplos:

Controllers terminam com Controller;

Casos de uso terminam com UseCase;

Serviços de domínio terminam com Service;

Repositórios terminam com Repository;

DTOs terminam com Request ou Response;

Mapeadores terminam com Mapper.

Capítulo 2 – Estrutura do Projeto e Organização dos Pacotes

2.1 Objetivo

Este capítulo define a estrutura física do projeto backend do Gestor de Territórios e Publicações (GTP).

Seu objetivo é estabelecer uma organização consistente para o código-fonte, facilitando:

manutenção;

evolução;

reutilização;

testes;

escalabilidade;

integração entre equipes.

A estrutura foi projetada para refletir diretamente o domínio do negócio definido nos documentos anteriores.

2.2 Estrutura Geral do Projeto

O backend será organizado como um projeto Maven.

gtp-backend

■

■■■ .mvn/

■■■ docker/

■■■ docs/

■■■ scripts/

- src/
- target/
- pom.xml
- README.md
- .gitignore

Descrição dos diretórios

DiretórioFinalidade.mvnConfiguração do Maven WrapperdockerArquivos Docker e Docker
ComposedocsDocumentação técnica do backendscriptsScripts
auxiliaressrcCódigo-fontetargetArtefatos gerados pelo Maven2.3 Organização do Código-Fonte

A estrutura principal seguirá o padrão do Maven.

src

-
- main
 - ■■■ java
 - ■■■ resources
-
- test
 - java
 - resources

2.4 Pacote Base

Todo o código da aplicação será organizado sob um pacote raiz.

br.com.gtp

A partir dele serão organizados os módulos de negócio e componentes compartilhados.

2.5 Organização dos Pacotes

br.com.gtp

- config
- security
- shared
-
- auth
- congregation
- people
- users
- territories
- publications
- orders
- campaigns
- notifications
- reports

■■■ settings

■■■ audit

Cada módulo será independente e conterá todos os elementos necessários para sua operação.

2.6 Estrutura Interna dos Módulos

Todos os módulos seguirão exatamente a mesma organização.

Exemplo utilizando o módulo Territórios.

territories

■■■ application

■

■■■ domain

■

■■■ infrastructure

■

■■■ interfaces

Essa padronização facilita a navegação e reduz a curva de aprendizado.

2.7 Camada Application

Responsável pela implementação dos casos de uso.

Estrutura:

application

■■■ dto

■■■ mapper

■■■ service

■■■ usecase

■■■ validator

Responsabilidades

DTO

Objetos utilizados na comunicação entre API e aplicação.

Exemplos:

CreateTerritoryRequest

UpdateTerritoryRequest

TerritoryResponse

Mapper

Conversão entre:

Entity

DTO

Response

Utilizará MapStruct.

Service

Coordena fluxos de aplicação.

Não contém regras de domínio complexas.

Use Case

Implementa diretamente os casos de uso descritos no Documento 04.

Exemplos:

ReservarTerritorioUseCase

DevolverTerritorioUseCase

CriarPedidoUseCase

EntregarPedidoUseCase

Validator

Valida regras específicas da aplicação antes da execução dos casos de uso.

2.8 Camada Domain

Representa o núcleo do negócio.

domain

■■■ entity

■■■ event

■■■ exception

■■■ repository

■■■ service

■■■ valueobject

Entity

Representa entidades do domínio.

Exemplos:

Congregation

Person

Territory

Publication

Order

Repository

Interfaces responsáveis pelo acesso aos dados.

Não conhecem JPA.

Exemplo:

```
public interface TerritoryRepository {  
}
```

Service

Serviços de domínio.

Implementam regras que envolvem múltiplas entidades.

Event

Eventos importantes do domínio.

Exemplos:

TerritoryReserved

OrderDelivered

StockUpdated

Exception

Exceções específicas do domínio.

Exemplos:

TerritoryUnavailableException

InsufficientStockException

InvalidOrderException

ValueObject

Objetos de valor.

Exemplos:

Address

Email

Phone

TerritoryCode

2.9 Camada Infrastructure

Implementa detalhes técnicos.

infrastructure

■■■ persistence

■■■ integration

■■■ configuration

■■■ external

Persistence

Implementações JPA.

Exemplos:

JpaTerritoryRepository

JpaOrderRepository

Integration

Comunicação com serviços externos.

Exemplos futuros:

envio de e-mails;

notificações;

armazenamento de arquivos;

integrações administrativas.

Configuration

Configurações específicas do módulo.

External

Clientes para APIs externas, quando necessário.

2.10 Camada Interfaces

Responsável pela exposição dos serviços.

interfaces

■■■■ controller

■■■■ advice

■■■■ documentation

Controller

Exposição dos endpoints REST.

Exemplo:

TerritoryController

OrderController

Advice

Tratamento centralizado de exceções do módulo.

Documentation

Documentação OpenAPI específica do módulo.

2.11 Pacote Shared

Este pacote conterá recursos reutilizados por toda a aplicação.

shared

■■■■ audit

■■■■ exception

■■■■ mapper

■■■■ pagination

■■■■ response

■■■■ validation

■■■■ util

■■■■ constants

2.12 Pacote Config

Centralizará configurações da aplicação.

Exemplos:

Beans

OpenAPI

CORS

Jackson

TimeZone

Locale

2.13 Pacote Security

Estrutura proposta:

security

■■■ jwt

■■■ filter

■■■ authentication

■■■ authorization

■■■ configuration

■■■ userdetails

Esse pacote será aprofundado no Documento 08.

2.14 Recursos (resources)

resources

application.yml

application-dev.yml

application-homolog.yml

application-prod.yml

db

migration

messages

static

application.yml

Configurações comuns.

Profiles

Separação por ambiente:

desenvolvimento;

homologação;

produção.

Flyway

db

migration

V1__Initial.sql

V2__Users.sql

V3__Territories.sql

...

Todas as alterações estruturais do banco serão versionadas.

2.15 Estrutura de Testes

test

java

resources

integration

unit

Separação entre:

testes unitários;

testes de integração.

2.16 Convenções de Nomenclatura

ElementoConvençãoControllerTerritoryControllerServiceTerritoryServiceRepositoryTerritoryRepositoryDTOCreateTerritoryRequestResponseTerritoryResponseMapperTerritoryMapperUseCaseReserveTerritoryUseCaseExceptionTerritoryUnavailableException2.17 Dependências entre Camadas

A comunicação entre as camadas seguirá uma única direção.

Controller



Use Case



Domain Service



Repository Interface



JPA Repository



PostgreSQL

Nenhuma camada inferior poderá depender diretamente de uma camada superior, preservando a independência do domínio.

2.18 Boas Práticas de Organização

Para manter a consistência do projeto, deverão ser observadas as seguintes diretrizes:

um módulo não deve acessar diretamente a infraestrutura de outro módulo;

regras de negócio devem permanecer na camada de domínio;

controladores devem apenas receber requisições e delegar aos casos de uso;

evitar classes utilitárias excessivamente genéricas;

utilizar interfaces para abstração sempre que houver dependência entre componentes;

manter módulos pequenos e coesos.

2.19 Exemplo de Organização Completa de um Módulo

territories/

■ ■ ■ application/

■ ■ ■ ■ dto/

■ ■ ■ ■ mapper/

■ ■ ■ ■ service/

■ ■ ■ ■ usecase/

■ ■ ■ ■ validator/

■ ■ ■ domain/

■ ■ ■ ■ entity/

■ ■ ■ ■ event/

■ ■ ■ ■ exception/

■ ■ ■ ■ repository/

■ ■ ■ ■ service/

■ ■ ■ ■ valueobject/

■ ■ ■ infrastructure/

■ ■ ■ ■ configuration/

■ ■ ■ ■ integration/

■ ■ ■ ■ persistence/

■ ■ ■ ■ external/

■ ■ ■ interfaces/

■ ■ ■ advice/

■ ■ ■ controller/

■ ■ ■ documentation/

Essa estrutura será repetida para todos os módulos do GTP, garantindo uniformidade e facilitando a manutenção.

Capítulo 3 – Camada de Apresentação (API REST e Controllers)

3.1 Objetivo

A camada de apresentação representa a porta de entrada do backend do Gestor de Territórios e Publicações (GTP).

Sua responsabilidade é:

receber requisições HTTP;

validar parâmetros básicos;

autenticar e autorizar o acesso;

encaminhar a execução para os Casos de Uso;

retornar respostas padronizadas.

Nenhuma regra de negócio deverá ser implementada diretamente nesta camada.

3.2 Responsabilidades da Camada

Os Controllers deverão executar apenas as seguintes atividades:

receber requisições;

validar dados de entrada;
converter DTOs;
chamar os Casos de Uso;
retornar códigos HTTP apropriados;
documentar os endpoints;
registrar informações de auditoria quando aplicável.

Não é permitido que Controllers:

acessem diretamente o banco de dados;
implementem regras de negócio;
manipulem entidades de persistência diretamente;
executem cálculos de domínio.

3.3 Organização dos Controllers

Cada módulo possuirá seus próprios Controllers.

interfaces/

■■■■ controller/

■■■■ AuthController

■■■■ CongregationController

■■■■ PersonController

■■■■ UserController

■■■■ TerritoryController

■■■■ PublicationController

■■■■ OrderController

■■■■ CampaignController

■■■■ NotificationController

■■■■ ReportController

■■■■ SettingsController

■■■■ AuditController

Essa organização acompanha a divisão por módulos definida na arquitetura.

3.4 Convenções de Endpoints

Todos os endpoints seguirão convenções REST.

Exemplos

Congregações

GET /api/v1/congregations

GET /api/v1/congregations/{id}

POST /api/v1/congregations

PUT /api/v1/congregations/{id}

DELETE /api/v1/congregations/{id}

Territórios

GET /api/v1/territories

GET /api/v1/territories/{id}

POST /api/v1/territories

PUT /api/v1/territories/{id}

DELETE /api/v1/territories/{id}

Publicações

GET /api/v1/publications

POST /api/v1/publications

PUT /api/v1/publications/{id}

DELETE /api/v1/publications/{id}

Pedidos

POST /api/v1/orders

GET /api/v1/orders

GET /api/v1/orders/{id}

PUT /api/v1/orders/{id}

Além das operações CRUD, existirão endpoints específicos para ações de negócio, como reservar territórios, devolver territórios e aprovar pedidos.

3.5 Versionamento da API

Todos os endpoints serão versionados.

Formato:

/api/v1/

Exemplo:

/api/v1/territories

Essa estratégia permite evoluir a API mantendo compatibilidade com clientes existentes.

3.6 Fluxo de Processamento

O fluxo padrão de uma requisição será:

Cliente



Controller



DTO Request



Use Case



Domínio



Repository



Banco de Dados



DTO Response



Controller



Cliente

Cada camada possui responsabilidades bem definidas.

3.7 DTOs (Data Transfer Objects)

Os Controllers utilizarão exclusivamente DTOs para entrada e saída de dados.

Tipos de DTO

Request DTO

Representa os dados enviados pelo cliente.

Exemplos:

CreateTerritoryRequest

UpdatePublicationRequest

CreateOrderRequest

Response DTO

Representa a resposta enviada ao cliente.

Exemplos:

TerritoryResponse

PublicationResponse

OrderResponse

Summary DTO

Utilizado em listagens resumidas.

Exemplos:

TerritorySummaryResponse

PublicationSummaryResponse

Detail DTO

Utilizado para consultas completas.

Exemplos:

TerritoryDetailResponse

OrderDetailResponse

3.8 Padrão das Respostas

As respostas da API seguirão um formato uniforme.

Resposta de sucesso

```
{
  "success": true,
  "message": "Operação realizada com sucesso.",
  "data": {}
}
```

Resposta de erro

```
{
  "success": false,
  "message": "Território indisponível.",
  "errors": [
    "O território já está reservado."
  ]
}
```

Essa padronização simplifica o consumo pelo frontend.

3.9 Códigos HTTP

CódigoSituação200Operação realizada com sucesso201Recurso criado204Operação sem conteúdo de retorno400Requisição inválida401Não autenticado403Sem permissão404Recurso não encontrado409Conflito de estado422Erro de validação500Erro interno3.10 Validação de Entradas

Os Controllers realizarão apenas validações estruturais, utilizando Jakarta Validation.

Exemplos:

campos obrigatórios;

tamanho máximo;

formato de e-mail;

datas válidas;

limites numéricos.

Regras de negócio permanecerão nos Casos de Uso e no Domínio.

3.11 Tratamento de Exceções

As exceções serão tratadas de forma centralizada por um GlobalExceptionHandler.

Categorias:

validação;

autenticação;

autorização;
recurso não encontrado;
conflito de estado;
regra de negócio;
erro interno.

As mensagens retornadas ao cliente serão claras e não exporão detalhes internos da aplicação.

3.12 Documentação da API

Todos os Controllers serão documentados utilizando OpenAPI / Swagger.

A documentação deverá apresentar:

descrição do endpoint;
parâmetros;
exemplos de requisição;
exemplos de resposta;
códigos HTTP possíveis;
requisitos de autenticação.

Essa documentação servirá tanto para o frontend quanto para integrações futuras.

3.13 Paginação

Listagens utilizarão paginação padronizada.

Parâmetros previstos:

page
size
sort
direction

Exemplo:

GET /api/v1/publications?page=0&size=20&sort=name&direction=asc

3.14 Filtros e Pesquisas

As consultas permitirão filtros por critérios específicos.

Exemplos:

nome;
código;
status;
data;
congregação;
responsável.

Filtros poderão ser combinados para consultas mais precisas.

3.15 Boas Práticas para Controllers

Os Controllers deverão:

possuir apenas uma responsabilidade;

ser pequenos e objetivos;
delegar processamento aos Casos de Uso;
utilizar DTOs;
não conter lógica de negócio;
não acessar diretamente repositórios.

3.16 Fluxo Simplificado

HTTP Request



Controller



DTO



Use Case



Serviço de Domínio



Repository



Banco de Dados



Response DTO



HTTP Response

3.17 Integração com o Frontend

Os Controllers serão consumidos exclusivamente pela aplicação frontend do GTP.

A comunicação seguirá os padrões definidos no Documento 07 – Arquitetura Frontend e no Documento 10 – API REST.

Os contratos estabelecidos deverão permanecer estáveis e versionados.

3.18 Relação com os Casos de Uso

Cada endpoint será associado a um Caso de Uso documentado no Documento 04 – Casos de Uso.

Exemplos:

EndpointCaso de UsoPOST /ordersCriar PedidoPOST /territories/{id}/reserveReservar TerritórioPOST /territories/{id}/returnRegistrar Devolução de TerritórioPOST /orders/{id}/deliverEntregar PedidoGET /reports/publicationsEmitir Relatório de Publicações

Essa rastreabilidade garante alinhamento entre requisitos, implementação e documentação.

Capítulo 4 – Camada de Aplicação (Casos de Uso e Serviços de Aplicação)

4.1 Objetivo

A Camada de Aplicação é responsável por implementar os processos de negócio descritos no Documento 04 – Casos de Uso.

Ela coordena a execução das operações, controla transações, valida pré-condições e integra os componentes necessários para realizar cada funcionalidade do sistema.

Seu papel é orquestrar o fluxo da aplicação sem assumir responsabilidades do domínio ou da infraestrutura.

4.2 Responsabilidades da Camada

A Camada de Aplicação deverá:

- implementar os Casos de Uso;
- coordenar serviços de domínio;
- controlar transações;
- validar regras de aplicação;
- converter DTOs em objetos de domínio;
- publicar eventos de aplicação;
- acionar integrações quando necessário;
- retornar respostas para a camada de apresentação.

Não é responsabilidade desta camada:

- executar consultas SQL;
- implementar regras específicas das entidades;
- gerar respostas HTTP;
- controlar autenticação.

4.3 Estrutura da Camada

application

■

■■■ dto

■■■ mapper

■■■ service

■■■ usecase

■■■ validator

■■■ event

Cada pacote possui responsabilidades específicas.

4.4 Casos de Uso

Cada funcionalidade documentada será implementada por um Caso de Uso dedicado.

Exemplos:

CreateCongregationUseCase

UpdateCongregationUseCase

CreateUserUseCase

UpdateUserUseCase

ReserveTerritoryUseCase

ReturnTerritoryUseCase

CreatePublicationOrderUseCase

DeliverPublicationOrderUseCase

CreateCampaignUseCase

CloseCampaignUseCase

Cada Caso de Uso deverá representar uma única operação de negócio.

4.5 Estrutura de um Caso de Uso

Todos os Casos de Uso seguirão um padrão uniforme.

UseCase

■

■■■ validar entrada

■■■ consultar domínio

■■■ executar regras

■■■ persistir alterações

■■■ publicar eventos

■■■ retornar resultado

Esse fluxo garante consistência entre todas as funcionalidades.

4.6 Serviços de Aplicação

Os Serviços de Aplicação coordenam múltiplos Casos de Uso ou operações relacionadas.

Exemplos:

TerritoryApplicationService

OrderApplicationService

PublicationApplicationService

Esses serviços:

organizam operações complexas;

reutilizam Casos de Uso;

evitam duplicação de código;

centralizam fluxos recorrentes.

4.7 Controle Transacional

Todas as operações que alteram o estado do sistema deverão ser executadas em transações.

Exemplos:

criação de pedidos;

entrega de publicações;
retirada de territórios;
devolução de territórios;
atualização de estoque;
cadastro de usuários.

Caso qualquer etapa falhe, toda a transação deverá ser revertida.

4.8 Fluxo de Execução

Controller



Use Case



Validação



Serviço de Domínio



Repository



Banco de Dados



Eventos



Response DTO

4.9 Validações da Aplicação

As validações desta camada verificam aspectos relacionados ao fluxo da operação.

Exemplos:

usuário autenticado;
perfil autorizado;
existência do recurso;
disponibilidade do território;
disponibilidade de estoque;
consistência da solicitação.

Regras específicas do domínio permanecem nas entidades e serviços de domínio.

4.10 Conversão de Objetos

A conversão entre DTOs e entidades será realizada por MapStruct.

Fluxo:

Request DTO



Mapper



Entity



Use Case



Mapper



Response DTO

Isso reduz código repetitivo e padroniza o mapeamento.

4.11 Eventos de Aplicação

Após determinadas operações, eventos poderão ser publicados.

Exemplos:

pedido criado;

pedido entregue;

território reservado;

território devolvido;

campanha iniciada;

campanha encerrada.

Esses eventos permitirão integrações futuras sem acoplamento direto.

4.12 Tratamento de Erros

Os Casos de Uso deverão lançar exceções específicas quando ocorrerem situações como:

recurso inexistente;

operação não permitida;

conflito de estado;

estoque insuficiente;

território indisponível;

usuário sem permissão.

As exceções serão tratadas posteriormente pelo mecanismo global definido na camada de apresentação.

4.13 Integração com o Domínio

A Camada de Aplicação nunca implementará regras próprias das entidades.

Fluxo:

Use Case



Entity



Domain Service

As decisões de negócio permanecerão concentradas no domínio.

4.14 Organização por Funcionalidade

Cada módulo possuirá seus próprios Casos de Uso.

Exemplo:

territories



■■■ application

■■■ usecase

■■■ CreateTerritoryUseCase

■■■ UpdateTerritoryUseCase

■■■ ReserveTerritoryUseCase

■■■ ReturnTerritoryUseCase

■■■ ArchiveTerritoryUseCase

■■■ DeleteTerritoryUseCase

A mesma estrutura será aplicada aos módulos de Publicações, Pedidos, Usuários, Campanhas e demais funcionalidades.

4.15 Dependências Permitidas

A Camada de Aplicação poderá depender de:

interfaces de repositório;

entidades;

serviços de domínio;

validadores;

mapeadores;

DTOs.

Não poderá depender diretamente de implementações da infraestrutura.

4.16 Padrões de Implementação

Cada Caso de Uso deverá:

representar apenas uma ação de negócio;

possuir nome claro;

ser pequeno;

facilitar testes;

evitar dependências desnecessárias;

utilizar injeção de dependência;

retornar objetos padronizados.

4.17 Testabilidade

Todos os Casos de Uso deverão possuir testes unitários.

Serão testados:

fluxo principal;

exceções;

validações;

cenários alternativos;

regras condicionais.

Dependências externas deverão ser simuladas por meio de mocks.

4.18 Exemplo de Fluxo Completo

Caso de Uso: Reservar Território

Usuário



POST /territories/{id}/reserve



TerritoryController



ReserveTerritoryUseCase



Validação



TerritoryService



TerritoryRepository



PostgreSQL



Evento TerritoryReserved



Response DTO

Esse fluxo demonstra como a camada de aplicação coordena todos os componentes envolvidos.

4.19 Relação com os Demais Documentos

As definições deste capítulo estão diretamente relacionadas a:

DocumentoRelaçãoDocumento 04 – Casos de UsoCada Caso de Uso implementa um fluxo funcional documentado.Documento 05 – Arquitetura do SistemaDefine a posição da camada de aplicação na arquitetura geral.Documento 10 – API RESTOs endpoints invocam os Casos de Uso descritos neste capítulo.4.20 Benefícios da Camada de Aplicação

A organização proposta proporciona:

separação clara entre interface e domínio;

maior reutilização de código;

facilidade para testes automatizados;

baixo acoplamento;

manutenção simplificada;

rastreabilidade entre requisitos e implementação;

preparação para integrações futuras.

Capítulo 5 – Camada de Domínio (Entidades, Objetos de Valor, Agregados e Serviços de Domínio)

5.1 Objetivo

A Camada de Domínio representa o núcleo do Gestor de Territórios e Publicações (GTP).

Ela concentra:

regras de negócio;

entidades;

objetos de valor;

agregados;

serviços de domínio;

eventos de domínio;

interfaces de repositório;

exceções específicas do negócio.

Nenhuma dependência de infraestrutura deverá existir nesta camada.

5.2 Responsabilidades

A Camada de Domínio será responsável por:

representar o negócio;
proteger invariantes;
impedir estados inválidos;
aplicar regras permanentes do sistema;
garantir consistência entre entidades relacionadas.

Não será responsável por:

persistência de dados;
comunicação HTTP;
autenticação;
envio de e-mails;
geração de relatórios;
acesso ao banco de dados.

5.3 Estrutura da Camada

domain



■■■ entity

■■■ valueobject

■■■ aggregate

■■■ repository

■■■ service

■■■ event

■■■ specification

■■■ exception

Cada pacote possui uma finalidade bem definida.

5.4 Entidades do Domínio

As entidades representam objetos que possuem identidade própria e ciclo de vida.

No GTP, as principais entidades são:

EntidadeResponsabilidadeCongregationRepresenta a congregação responsável pelo território e pelo estoque local de publicações. PersonRepresenta os dados cadastrais de uma pessoa. UserRepresenta o usuário autenticado do sistema. TerritoryRepresenta um território de pregação pertencente a uma congregação. TerritoryAssignmentControla retiradas, reservas e devoluções de territórios. PublicationRepresenta uma publicação cadastrada no sistema. StockControla o estoque de publicações por congregação. OrderRepresenta um pedido de publicações. OrderItemRepresenta os itens pertencentes a um pedido. CampaignRepresenta campanhas especiais de distribuição de publicações. NotificationRepresenta notificações emitidas pelo sistema. AuditLogRepresenta registros de auditoria das operações.

5.5 Características das Entidades

Todas as entidades deverão:

possuir identidade única;
controlar seu próprio estado;
proteger suas regras internas;

impedir alterações inválidas;

encapsular comportamento.

Exemplo:

Um Territory não poderá ser marcado como disponível se existir uma retirada ativa.

Essa validação pertence à própria entidade.

5.6 Objetos de Valor (Value Objects)

Objetos de Valor representam conceitos que não possuem identidade própria e são definidos exclusivamente por seus atributos.

No GTP poderão ser utilizados:

Value Object
FinalidadeAddressEndereço da congregação ou da pessoa.
EmailEndereço eletrônico validado.
PhoneNúmero telefônico padronizado.
TerritoryCodeCódigo único do território.
PublicationCodeCódigo da publicação.
QuantityQuantidade de publicações.
PeriodIntervalo de datas.

Características:

imutáveis;

comparados por valor;

sem identificador próprio.

5.7 Agregados (Aggregates)

Os agregados agrupam entidades relacionadas que devem permanecer consistentes entre si.

Principais agregados do GTP:

Congregation Aggregate

Congregation

■■■ Territories

■■■ Stock

■■■ Publications

■■■ Users

A Congregação atua como raiz desse conjunto de informações.

Order Aggregate

Order

■■■ OrderItems

■■■ DeliveryInformation

Toda alteração nos itens deverá ocorrer por intermédio do pedido.

Territory Aggregate

Territory

■■■ TerritoryAssignment

As reservas, retiradas e devoluções deverão ser controladas pela raiz do agregado.

5.8 Serviços de Domínio

Algumas regras envolvem múltiplas entidades e não pertencem naturalmente a apenas uma delas.

Nesses casos serão utilizados Serviços de Domínio.

Exemplos:

TerritoryService

PublicationService

OrderService

CampaignService

StockService

Responsabilidades:

coordenar regras entre entidades;

preservar consistência;

evitar duplicação de lógica.

5.9 Interfaces de Repositório

O domínio define apenas contratos.

Exemplo:

```
public interface TerritoryRepository {  
    Optional<Territory> findById(UUID id);  
    List<Territory> findAvailable();  
    Territory save(Territory territory);  
}
```

A implementação concreta ficará na camada de infraestrutura.

5.10 Eventos de Domínio

Eventos representam fatos importantes ocorridos no sistema.

Exemplos:

TerritoryReserved

TerritoryReturned

TerritoryArchived

OrderCreated

OrderDelivered

PublicationReceived

CampaignStarted

CampaignFinished

StockUpdated

Esses eventos permitem integração entre módulos mantendo baixo acoplamento.

5.11 Especificações (Specification Pattern)

Regras complexas poderão ser encapsuladas em Specifications.

Exemplos:

TerritoryAvailableSpecification

UserCanReceiveTerritorySpecification

StockAvailableSpecification

OrderCanBeDeliveredSpecification

Isso melhora reutilização e legibilidade.

5.12 Exceções de Domínio

As exceções representam violações das regras de negócio.

Exemplos:

TerritoryUnavailableException

InvalidTerritoryStateException

PublicationOutOfStockException

InvalidOrderException

CampaignClosedException

UserWithoutPermissionException

Cada exceção deverá possuir mensagem clara e específica.

5.13 Invariantes do Domínio

As invariantes representam regras que nunca poderão ser violadas.

Exemplos do GTP:

um território não pode estar atribuído simultaneamente a dois publicadores;

apenas usuários autorizados podem entregar pedidos;

o estoque não pode assumir quantidade negativa;

um pedido entregue não pode retornar ao estado pendente;

uma campanha encerrada não pode receber novos pedidos;

uma congregação não pode possuir territórios duplicados.

As entidades deverão impedir qualquer alteração que viole essas regras.

5.14 Modelo de Dependências

A Camada de Domínio não dependerá de frameworks.

Domain

■

■■■ Entity

■■■ ValueObject

■■■ Aggregate

■■■ Service

■■■ Repository (Interface)

■■■ Event

■■■ Exception

Ela será completamente independente do Spring Boot.

5.15 Fluxo Interno do Domínio

Use Case

■

▼

Entity



Domain Service



Repository Interface

A infraestrutura apenas implementará os contratos definidos pelo domínio.

5.16 Organização por Módulo

Cada módulo possuirá sua própria estrutura de domínio.

Exemplo:

territories

■■■ domain

■■■ entity

■ ■■■ Territory

■ ■■■ TerritoryAssignment

■■■ repository

■■■ service

■■■ event

■■■ exception

■■■ specification

■■■ valueobject

O mesmo padrão será aplicado aos módulos de Publicações, Pedidos, Usuários, Campanhas e Congregações.

5.17 Benefícios da Modelagem

A adoção de DDD proporciona:

isolamento das regras de negócio;

alta coesão;

baixo acoplamento;

facilidade para testes;

evolução incremental;

reutilização de comportamento;

maior alinhamento entre código e domínio.

5.18 Relação com os Documentos do Projeto

A Camada de Domínio implementa diretamente os conceitos definidos em toda a documentação produzida.

DocumentoRelaçãoDocumento 01 – Modelo Conceitual do DomínioDefine os conceitos representados pelas entidades e objetos de valor.Documento 02 – Requisitos FuncionaisDefine as funcionalidades implementadas pelos agregados e serviços.Documento 03 – Regras de NegócioDefine as invariantes protegidas pelo domínio.Documento 04 – Casos de UsoOs fluxos

operacionais são executados utilizando entidades e serviços de domínio.Documento 05 –
Arquitetura do SistemaDefine a posição da Camada de Domínio na arquitetura geral.5.19 Exemplo
Integrado de Fluxo

Caso de Uso: Retirar Território

ReserveTerritoryUseCase



Territory



TerritoryService



TerritoryAssignment



Domain Event



Repository Interface

Nesse fluxo, todas as decisões de negócio permanecem concentradas na Camada de Domínio.

5.20 Conclusão

A Camada de Domínio constitui o coração do backend do GTP. Ela reúne o conhecimento do negócio, protege as regras fundamentais da aplicação e garante que todas as operações sejam executadas de forma consistente e independente da tecnologia utilizada.

Ao concentrar entidades, objetos de valor, agregados, serviços, eventos e invariantes em uma camada desacoplada, a arquitetura assegura que futuras mudanças de infraestrutura não comprometam o comportamento do sistema.

Capítulo 6 – Camada de Infraestrutura (Persistência, Integrações e Recursos Técnicos)

6.1 Objetivo

A Camada de Infraestrutura é responsável por implementar todos os recursos tecnológicos necessários para o funcionamento do Gestor de Territórios e Publicações (GTP).

Ela conecta o domínio às tecnologias utilizadas pela aplicação, fornecendo implementações concretas para persistência, comunicação externa, auditoria, armazenamento e configurações.

O domínio permanece desacoplado dessas tecnologias por meio de interfaces e inversão de dependências.

6.2 Responsabilidades

A Camada de Infraestrutura será responsável por:

implementar os repositórios definidos no domínio;

realizar persistência de dados;

integrar o sistema ao PostgreSQL;

executar migrações de banco com Flyway;
integrar serviços externos;
gerenciar armazenamento de arquivos;
publicar e consumir eventos técnicos;
fornecer configurações da aplicação;
implementar mecanismos de cache, quando adotados.
Não é responsabilidade desta camada definir regras de negócio.

6.3 Estrutura da Camada

infrastructure

■

■■■ configuration

■■■ persistence

■■■ integration

■■■ external

■■■ storage

■■■ messaging

■■■ scheduler

■■■ audit

Cada pacote concentra implementações específicas da infraestrutura.

6.4 Persistência de Dados

A persistência será implementada com:

Spring Data JPA;

Hibernate;

PostgreSQL 17.

Cada interface de repositório definida na Camada de Domínio terá uma implementação correspondente nesta camada.

Exemplo:

domain/

■■■ repository/

■■■ TerritoryRepository

↓

infrastructure/

■■■ persistence/

■■■ JpaTerritoryRepository

Essa separação mantém o domínio independente da tecnologia de persistência.

6.5 Implementação dos Repositórios

Os repositórios concretos serão responsáveis por:

consultas ao banco de dados;

persistência de entidades;

paginação;

filtros;

consultas otimizadas;

execução de especificações.

A lógica de negócio permanecerá fora dos repositórios.

6.6 Mapeamento Objeto-Relacional

O Hibernate será utilizado para realizar o mapeamento entre entidades e tabelas do banco de dados.

As entidades deverão ser anotadas conforme os padrões da JPA, mantendo o mínimo de acoplamento possível.

Aspectos como relacionamentos, índices e restrições serão definidos de forma consistente com o modelo físico do banco de dados.

6.7 Migrações de Banco de Dados

A evolução do esquema será controlada pelo Flyway.

Estrutura:

resources/

■■■ db/

■■■ migration/

■■■ V1__Initial_Schema.sql

■■■ V2__Users.sql

■■■ V3__Congregations.sql

■■■ V4__Territories.sql

■■■ V5__Publications.sql

■■■ V6__Orders.sql

■■■ ...

Cada migração deverá ser:

incremental;

versionada;

imutável após aplicada.

6.8 Configurações da Aplicação

As configurações serão organizadas por ambiente.

resources/

■■■ application.yml

■■■ application-dev.yml

■■■ application-homolog.yml

■■■ application-prod.yml

As configurações incluirão:

conexão com banco;

logs;

autenticação;

auditoria;

internacionalização;

parâmetros gerais.

Informações sensíveis serão fornecidas por variáveis de ambiente ou mecanismos equivalentes.

6.9 Integrações Externas

A arquitetura permitirá futuras integrações com serviços externos, como:

envio de e-mails;

notificações;

armazenamento de documentos;

sistemas administrativos;

serviços de autenticação;

aplicações móveis.

Cada integração será encapsulada por interfaces e adaptadores específicos.

6.10 Armazenamento de Arquivos

O sistema poderá armazenar documentos relacionados ao processo administrativo, quando necessário.

Exemplos:

documentos de apoio;

arquivos de campanhas;

modelos de relatórios.

A implementação deverá permitir substituição do mecanismo de armazenamento sem impacto no domínio.

6.11 Auditoria Técnica

Além da auditoria funcional, a infraestrutura registrará eventos técnicos.

Exemplos:

inicialização da aplicação;

execução de migrações;

falhas de integração;

erros inesperados;

indisponibilidade de serviços externos.

Esses registros auxiliarão na manutenção e no diagnóstico do sistema.

6.12 Agendamento de Tarefas

Algumas rotinas poderão ser executadas automaticamente.

Exemplos:

atualização de indicadores;

encerramento automático de campanhas;

limpeza de dados temporários;

envio de notificações programadas;

geração periódica de relatórios.

Essas tarefas serão implementadas em componentes específicos da infraestrutura.

6.13 Eventos Técnicos

A infraestrutura poderá publicar ou consumir eventos assíncronos para desacoplar processos.

Exemplos:

atualização de estoque;

envio de notificações;

registro de auditoria;

sincronização entre módulos.

Essa abordagem favorece escalabilidade e manutenção.

6.14 Cache

Quando necessário, poderão ser utilizados mecanismos de cache para otimizar operações de leitura.

Possíveis aplicações:

configurações do sistema;

parâmetros gerais;

listas de referência;

dados pouco voláteis.

O uso de cache deverá preservar a consistência dos dados.

6.15 Organização por Módulo

Cada módulo poderá possuir implementações específicas de infraestrutura.

Exemplo:

territories/

■■■ infrastructure

■■■ configuration

■■■ persistence

■■■ integration

■■■ external

■■■ storage

Essa organização mantém o alinhamento com a arquitetura modular do projeto.

6.16 Comunicação entre Camadas

A infraestrutura será acessada apenas por meio das interfaces definidas no domínio.

Fluxo:

Use Case

■

▼

Repository (Interface)



JpaRepository



Hibernate



PostgreSQL

Essa inversão de dependência preserva a independência do domínio.

6.17 Tratamento de Exceções Técnicas

Exceções relacionadas à infraestrutura deverão ser tratadas separadamente das exceções de negócio.

Exemplos:

falha de conexão com banco de dados;

indisponibilidade de serviço externo;

erro de leitura ou gravação de arquivos;

timeout em integrações;

falhas de migração.

Esses erros serão registrados em logs e convertidos em respostas apropriadas para a camada de apresentação.

6.18 Testes da Infraestrutura

A infraestrutura deverá ser validada por meio de:

testes de integração com banco de dados;

testes de migrações Flyway;

testes de repositórios;

testes de clientes de integração;

testes de componentes agendados.

Sempre que possível, será utilizado Testcontainers para garantir ambientes de teste reproduzíveis.

6.19 Relação com os Demais Documentos

A Camada de Infraestrutura complementa as definições dos seguintes documentos:

Documento 05 – Arquitetura do Sistema Define a posição da infraestrutura na arquitetura geral.

Documento 08 – Segurança Configura autenticação, autorização e auditoria.

Documento 09 – Docker Define a implantação da infraestrutura em contêineres.

Documento 10 – API REST Consome os serviços disponibilizados pela infraestrutura.

Documentação de Banco de Dados Define o modelo físico implementado pelos repositórios.

6.20 Benefícios da Arquitetura

A organização proposta proporciona:

desacoplamento entre domínio e tecnologia;

facilidade de substituição de componentes técnicos;

- manutenção simplificada;
- maior testabilidade;
- escalabilidade;
- suporte a integrações futuras;
- rastreabilidade das operações técnicas;
- evolução controlada do banco de dados.

Conclusão

A Camada de Infraestrutura conecta o domínio do GTP aos recursos tecnológicos necessários para sua execução, preservando a independência das regras de negócio e garantindo uma implementação robusta e preparada para evolução.

Ao concentrar persistência, integrações, configurações, auditoria e demais recursos técnicos em uma camada específica, a arquitetura mantém a aplicação organizada, modular e aderente aos princípios definidos nos documentos anteriores.

Capítulo 7 – Segurança, Observabilidade, Testes e Diretrizes de Implementação

7.1 Objetivo

Este capítulo estabelece as diretrizes técnicas que deverão ser observadas em toda a implementação do backend do GTP.

Seu propósito é garantir que o sistema seja:

- seguro;
- confiável;
- observável;
- testável;
- escalável;
- padronizado;
- de fácil manutenção.

Essas diretrizes complementam os capítulos anteriores e servem como referência para todas as equipes envolvidas no desenvolvimento.

7.2 Arquitetura de Segurança

A segurança será implementada utilizando o Spring Security, com autenticação baseada em JWT (JSON Web Token) e autorização baseada em perfis (Roles) e permissões (Authorities).

A arquitetura deverá garantir:

- autenticação de usuários;
- autorização por perfil;
- proteção contra acessos não autorizados;
- integridade das informações;
- confidencialidade dos dados;
- rastreabilidade das ações.

O detalhamento completo será apresentado no Documento 08 – Segurança, mas a arquitetura do backend já deve ser preparada para suportar esses mecanismos.

7.3 Fluxo de Autenticação

O fluxo de autenticação seguirá o padrão abaixo:

Usuário



Login (usuário e senha)



Spring Security



Authentication Manager



Validação das credenciais



JWT



Cliente



Requisições autenticadas

Todas as requisições protegidas deverão incluir um Bearer Token válido.

7.4 Autorização

O acesso às funcionalidades será controlado por perfis e permissões.

Perfis previstos:

PerfilResponsabilidadesAdministrador GlobalAdministração completa do sistema.Administrador da CongregaçãoGestão da congregação e seus recursos.Servo MinisterialOperações administrativas autorizadas.AnciãoGestão de territórios, publicações e campanhas conforme permissões.PublicadorConsulta de informações e operações autorizadas.Usuário de ConsultaAcesso somente para leitura.

As permissões deverão ser verificadas antes da execução de cada Caso de Uso.

7.5 Proteção dos Endpoints

Todos os endpoints serão classificados conforme seu nível de acesso.

CategoriaExemploAutenticaçãoPúblico/api/v1/auth/loginNãoAutenticado/api/v1/profileSimAdministrativo/api/v1/usersSim + PerfilOperacional/api/v1/ordersSim + PermissãoAuditoria/api/v1/auditSim + Perfil específico

Essa classificação reduz a superfície de ataque e facilita a gestão de permissões.

7.6 Auditoria

Todas as operações relevantes deverão gerar registros de auditoria.

Informações mínimas registradas:

usuário responsável;
data e hora;
operação executada;
módulo;
entidade afetada;
identificador do registro;
resultado da operação;
endereço IP (quando aplicável);
identificador da sessão.

Os registros de auditoria serão imutáveis e destinados à rastreabilidade das ações.

7.7 Observabilidade

O backend deverá fornecer mecanismos que permitam acompanhar o comportamento da aplicação em tempo real.

Os principais pilares são:

logs estruturados;
métricas de desempenho;
monitoramento de saúde da aplicação;
rastreamento de requisições.

Esses recursos facilitam a identificação de problemas e o acompanhamento da operação em produção.

7.8 Estratégia de Logs

Os logs deverão seguir um padrão consistente.

Categorias:

TipoObjetivoINFOOperações executadas normalmente.WARNSituações inesperadas sem impacto crítico.ERRORErros que impedem a conclusão da operação.DEBUGInformações detalhadas para desenvolvimento.

Dados sensíveis, como senhas e tokens, nunca deverão ser registrados em logs.

7.9 Monitoramento

A aplicação deverá disponibilizar mecanismos para monitoramento operacional.

Itens monitorados:

disponibilidade da aplicação;
tempo de resposta;
consumo de memória;
utilização de CPU;
conexões com banco de dados;
filas e tarefas agendadas;
integrações externas.

Essas informações poderão ser integradas a ferramentas de observabilidade adotadas pela organização.

7.10 Estratégia de Testes

O backend será validado por meio de uma estratégia de testes em múltiplas camadas.

Tipo de TesteObjetivoUnitárioValidar classes isoladamente.IntegraçãoValidar comunicação entre componentes.PersistênciaValidar acesso ao banco de dados.APIValidar endpoints REST.SegurançaValidar autenticação e autorização.PerformanceAvaliar comportamento sob carga.

Todos os testes deverão ser executados automaticamente durante o processo de integração contínua.

7.11 Cobertura de Testes

Os testes deverão contemplar:

fluxos principais;

fluxos alternativos;

validações;

exceções;

regras de negócio;

autenticação;

autorização;

integração entre módulos.

A cobertura mínima recomendada para a lógica de negócio é de 80%, priorizando os Casos de Uso e os Serviços de Domínio.

7.12 Integração Contínua

O backend deverá ser preparado para execução em pipelines de CI/CD.

O processo automatizado incluirá:

compilação do projeto;

análise estática de código;

execução dos testes;

validação das migrações do banco;

geração da documentação;

criação da imagem Docker;

publicação dos artefatos.

Nenhuma alteração deverá ser integrada à branch principal sem aprovação do pipeline.

7.13 Qualidade de Código

O desenvolvimento deverá seguir princípios de engenharia de software amplamente reconhecidos.

Diretrizes:

responsabilidade única (SRP);

aberto para extensão e fechado para modificação (OCP);

substituição de Liskov (LSP);

segregação de interfaces (ISP);

inversão de dependência (DIP).

Também deverão ser observados:

Clean Code;

Clean Architecture;

Domain-Driven Design (DDD);

boas práticas do ecossistema Spring.

7.14 Convenções de Desenvolvimento

Todos os módulos deverão seguir convenções uniformes.

Principais diretrizes:

nomes de classes em inglês;

nomenclatura consistente para métodos;

um Caso de Uso por classe;

DTOs separados de entidades;

utilização de injeção de dependências;

ausência de lógica de negócio em Controllers;

separação entre domínio e infraestrutura.

Essas convenções reduzem a complexidade e facilitam a colaboração entre desenvolvedores.

7.15 Gerenciamento de Configurações

As configurações da aplicação deverão ser externalizadas.

Princípios:

uso de perfis por ambiente;

variáveis de ambiente para dados sensíveis;

versionamento das configurações não confidenciais;

documentação de parâmetros obrigatórios.

Essa abordagem facilita a implantação em diferentes ambientes.

7.16 Escalabilidade

A arquitetura deverá permitir evolução gradual do sistema.

Aspectos considerados:

inclusão de novos módulos;

crescimento da base de usuários;

aumento do volume de dados;

novas integrações;

expansão para múltiplas congregações ou regiões.

O desacoplamento entre módulos reduz o impacto de futuras evoluções.

7.17 Checklist para Implementação

Antes da conclusão de cada funcionalidade, deverá ser verificado:

Caso de Uso implementado;

regras de negócio respeitadas;

testes unitários criados;

testes de integração executados;

documentação atualizada;
auditoria implementada;
tratamento de exceções realizado;
logs adicionados;
endpoint documentado;
revisão de código concluída.

Esse checklist padroniza a entrega de novas funcionalidades.

7.18 Rastreabilidade

Cada componente implementado deverá possuir correspondência com os documentos do projeto.

DocumentoArtefato RelacionadoDocumento 01 – Modelo Conceitual do DomínioEntidades, Objetos de Valor e Agregados.Documento 02 – Requisitos FuncionaisFuncionalidades implementadas pelos Casos de Uso.Documento 03 – Regras de NegócioRegras implementadas na Camada de Domínio.Documento 04 – Casos de UsoCasos de Uso da Camada de Aplicação.Documento 05 – Arquitetura do SistemaOrganização arquitetural geral.Documento 06 – Arquitetura BackendEstrutura técnica detalhada do backend.Documento 10 – API RESTEndpoints expostos pelos Controllers.

Essa rastreabilidade facilita auditorias, manutenção e evolução do sistema.

7.19 Considerações Finais

A arquitetura apresentada neste documento fornece uma base sólida para o desenvolvimento do backend do GTP, conciliando boas práticas de engenharia de software com uma organização modular e orientada ao domínio.

A adoção de Java 21, Spring Boot 3, DDD, Clean Architecture, Spring Security, PostgreSQL e Flyway assegura uma plataforma preparada para crescimento, manutenção de longo prazo e integração com futuras funcionalidades.